

Project Title

KAAVISH PROJECT PROPOSAL

By

Abdullah Shaikh (09245@st.habib.edu.pk)
Salman Kamal (sk08996@st.habib.edu.pk)
Taha Zahid (tz09220@st.habib.edu.pk)
Zain Naqi (09224@st.habib.edu.pk)

September 27, 2026



In partial fulfillment of the requirement for
Bachelor of Science
Computer Science

DHANANI SCHOOL OF SCIENCE AND ENGINEERING

HABIB UNIVERSITY

FALL 2023

Copyright © 2023 Habib University

1 Abstract

Robots are increasingly used to perform complex tasks in warehouses, hospitals, construction sites, and disaster response scenarios. In many of these situations, a single robot is not enough. A task might be too large, require different types of equipment, or need to be finished quickly. This is where multi-robot systems come in, where teams of robots work together toward a shared goal. Coordinating that team, though, is not simple. Deciding which robot should do which task, in what order, and by when is a hard problem in itself. This problem is called Multi-Robot Task Allocation, or MRTA.

For a long time, MRTA systems were programmed for specific scenarios. A team of robots built to sort packages in a warehouse could do exactly that, but nothing more. If the mission changed or an unexpected situation came up, the system had no way to handle it. The robots could only do what they were explicitly programmed for.

Large Language Models changed what was possible here. These models, trained on large amounts of text, can read natural language instructions, reason about goals, and break a complex task into smaller steps. Researchers started using them in multi-robot systems so that a human could describe a mission in plain language and have the robots figure out how to handle it. This opened up a lot of new possibilities. The problem is that most of these systems also use the LLM to decide which robot gets which task. That step, deciding who does what, is a mathematical problem. It requires finding an assignment that satisfies constraints around robot capabilities, travel distances, timing, and whether some tasks need to be done simultaneously by multiple robots. LLMs were not built for this and do not handle it well. Research has shown that when asked to make these decisions, they tend to produce assignments that are unbalanced, infeasible, or far from optimal, even when given the chance to review and correct their own output.

This project takes a different approach. The LLM is used only for the parts of the problem it handles well: reading the mission description, breaking it into subtasks, figuring out which subtasks depend on which others, and determining what type of coordination problem the mission represents. Once the problem type is identified, a dedicated mathematical solver handles the actual assignment of robots to tasks. Different types of problems have different mathematical structures, and there are well-established algorithms for each. Using the right solver for each problem type produces better results than asking the LLM to figure everything out.

We test this system in a simulated environment using a mixed fleet of wheeled ground robots, robotic arms, and aerial drones, all with different capabilities. The simulation covers seven different problem types, from straightforward cases where robots independently handle separate tasks to more involved scenarios where groups of robots need to coordinate in a particular order. We compare the results against systems that leave the allocation step to the LLM, using measures like total mission time, travel distance, and how far the solution is from the best possible assignment.

The aim is to show that keeping task allocation as a mathematical problem, and using the LLM only for language understanding, leads to more dependable outcomes. If this holds up, it could make multi-robot systems more practical in settings where the missions vary and cannot all be anticipated in advance.

2 Problem definition

Multi-Robot Task Allocation (MRTA) is the problem of computing an assignment from a set of tasks $\mathcal{T}$ to a team of heterogeneous robots $\mathcal{R}$ such that a global objective, typically total utility

or mission completion time, is improved subject to physical and operational constraints [1]. The difficulty of this problem is not the same in every scenario; it depends on the structure of the robots, the tasks, and the dependencies among them.

Gerkey and Mataric [1] formalized MRTA through a three-axis taxonomy. The first axis distinguishes Single-Task robots (ST), which execute at most one task at a time, from Multi-Task robots (MT). The second distinguishes Single-Robot tasks (SR), which require one robot, from Multi-Robot tasks (MR), which require a group of robots with complementary capabilities. The third distinguishes Instantaneous Assignment (IA), where assignment is made using information available at that moment, from Time-Extended Assignment (TA), where a schedule is built over a planning period. These combinations lead to different allocation and scheduling problems.

Korsah et al. [2] identified a limitation in this taxonomy: it assumes that the value of assigning a robot to a task is independent of other assignments. Their extended taxonomy, iTax, adds a dependency dimension. In No-Dependency (ND) problems, task assignments are independent. In In-Schedule Dependency (ID) problems, a robot's cost depends on the sequence of its own tasks, such as travel distance or total travel time. In Cross-Schedule Dependency (XD) problems, the schedules of different robots are linked through requirements such as precedence, time synchronization, or coalition work. This project targets seven subclasses: ND[ST-SR-IA], ND[ST-SR-TA], ID[ST-SR-TA], XD[ST-SR-IA], XD[ST-SR-TA], XD[ST-MR-IA], and XD[ST-MR-TA].

A related challenge is the interface between human operators and these systems. Traditional MRTA systems require missions to be given as structured inputs, which limits them to tasks they were explicitly programmed to handle. Large Language Models (LLMs) allow a user to describe a mission in natural language and can break it into smaller tasks. However, many LLM-based systems also use the LLM to decide which robot should perform each task. This is difficult because task allocation requires combinatorial reasoning over numerical constraints. LLMs can produce suboptimal or infeasible allocations in these cases [3].

This project addresses two related problems. First, how can a natural-language mission be interpreted, broken into a structured set of subtasks with dependencies, and classified into one of the iTax subclasses? Second, given that classification, which optimisation method should be used and how should its result be dispatched to a heterogeneous robot fleet? The key idea is that language understanding and combinatorial optimisation require different tools. The LLM is used for mission interpretation, while a specialised solver performs task allocation.

3 Social relevance

Multi-robot systems can support work that is important to people. In hospitals, they can move medicine and equipment. In disasters, they can enter unsafe areas to search for survivors or deliver supplies. In warehouses and farms, they can help move goods and monitor large areas. These uses can save time, reduce physical workload, and keep people away from dangerous tasks.

For these benefits to be useful, the robots must be given suitable work and must work together safely. A poor decision can delay urgent supplies, leave an important task unfinished, or send the wrong robot to a job. This project supports more reliable coordination of robot teams. In the long term, this can help organisations use robots more safely and effectively in services that affect the public.

4 Originality/Novelty

Classical MRTA systems use hand-crafted allocation methods, such as auctions, greedy matching, or fixed optimisation formulations. Systems such as ALLIANCE, BLE, and MURDOCH are well studied and give predictable results, but they require structured mission inputs and are built around a fixed allocation method [1]. They cannot interpret a mission given in natural language or adapt their allocation method when the problem structure changes.

LLM-based multi-robot systems address the natural-language limitation, but often use the LLM for both mission interpretation and task allocation. This is a problem because allocation must satisfy capability, timing, and dependency constraints. LLMs can understand a mission, but they are not designed to solve these combinatorial problems. As a result, they can produce infeasible or suboptimal allocations, even when asked to check their own answers [3].

Recent hybrid systems pair an LLM with a downstream solver. For example, LiP-LLM extracts task structure before solving a linear program, while DART-LLM turns a mission into a task DAG for a navigation system [3]. These are useful steps, but each system is tied to one optimisation model or domain. They do not first identify the structure of the allocation problem and then select the appropriate solver.

This project adds a taxonomy-classification layer between the LLM and the optimisation engine. The LLM converts a natural-language mission into a task structure and classifies it into one of seven iTax subclasses [2]. The system then routes the mission to a solver designed for that class, from simple assignment methods to coalition scheduling. This approach combines the flexibility of natural-language missions with an allocation method that matches the problem structure.

5 CS contribution

This project requires knowledge from several areas of Computer Science. The first area is artificial intelligence and natural language processing. We will build an LLM-based planning component with retrieval-augmented generation (RAG). It will retrieve the robot capability library, map information, and task-allocation taxonomy, and use this context to extract subtasks, target locations, required capabilities, estimated durations, and task dependencies from a natural-language mission. The component will produce a schema-constrained JSON plan that is parsed and validated by program code before it can be used by later modules. This requires work on prompt design, retrieval and indexing, context selection, structured output generation, and error handling for incomplete or invalid plans.

The second area is algorithms and graph-based modelling. The mission will be represented as a Directed Acyclic Graph (DAG), where each node is a subtask and each edge shows that one task must be completed before another. We will validate the graph and use its dependency information when planning the task order. This applies core concepts from data structures and graph algorithms to a practical robotics problem.

The main technical contribution is combinatorial optimisation. Each type of task-allocation problem will be mapped to a suitable mathematical model, such as an assignment problem, a routing problem, or a coalition-formation problem. Simple cases can be solved exactly using methods such as the Hungarian algorithm. More difficult cases, including variants of the Travelling Salesperson Problem, require integer programming or heuristic methods because they are computationally hard. This work draws directly on the optimisation and heuristic-search concepts studied in CS 451: Computational Intelligence.

Finally, the project includes robotics software integration and experimental evaluation. We will implement ROS nodes that convert the selected schedule into robot-specific action commands. These nodes will call the navigation, manipulation, or control interfaces needed for each subtask, monitor whether commands have been accepted or completed, and coordinate the order in which robots receive their work. The LLM provides the high-level mission plan; the ROS control code and robot controllers carry out movement and task execution in Gazebo. We will then measure mission time, travel distance, computation time, and solution quality to compare the system with baseline approaches under the same simulated conditions.

6 Scope and Deliverables

The project will be completed over one academic year in five stages. It is limited to a mapped, simulated environment with reliable communication and open-loop execution. We will use existing ROS and Gazebo robot-control interfaces rather than develop new low-level controllers. This keeps the work focused on task planning, allocation, integration, and evaluation.

The following are the planned deliverables:

1. **Environment plan, 2D test representation, and mission set:** By the end of the first stage, we will define the robot fleet, task stations, map layout, and task requirements. We will prepare at least one natural-language mission for each of the seven selected iTax classes and represent them in a 2D test setting.
2. **Mission-planning and classification module:** By the end of the second stage, we will implement an LLM and RAG module that converts each test mission into a validated JSON task plan. Each plan will include subtasks, capabilities, locations, and DAG dependencies, and will be assigned one of the seven target classes. It will be tested in the 2D setting, with limited ROS or Gazebo checks where needed.
3. **Task-allocation solvers:** In parallel, we will implement or adapt one suitable solver for each of the seven classes. For every standard mission, the solver must return a feasible robot assignment and schedule that meets its capability, dependency, and timing constraints.
4. **Integrated ROS–Gazebo pipeline:** By the fourth stage, we will build the 3D Gazebo environment and connect the validated mission plan, solver-routing layer, and ROS execution nodes. The pipeline will accept a test mission, select a solver, create a schedule, and send the resulting actions to the simulated robots.
5. **Benchmark and evaluation report:** In the final stage, we will compare the full pipeline with selected baselines, including direct LLM planning and greedy allocation where applicable. We will report classification accuracy, schedule feasibility, mission completion time, travel distance, solver time, and optimality gap when an optimal solution is available.

7 Feasibility

The project relies on a robotics stack built around ROS 2, Gazebo, MoveIt2, Nav2, and an LLM-based planning layer using Python and NetworkX. The seven iTax subclasses are formally defined

by Korsah et al. and each maps to a well-studied mathematical formulation with known algorithmic solutions. Solvers including the Hungarian algorithm, CVRP, and MILP-based methods are available through mature Python libraries such as SciPy, OR-Tools, and PuLP, so the solver layer will require adaptation and integration of existing tools rather than novel algorithm design.

The knowledge resources the system will depend on, including the robot capability registry, skill banks, and world map, will be constructed by the team as structured JSON and YAML files. The seven standard missions will be authored by the team and do not require external datasets. Since the entire system will run in simulation, no physical hardware is needed. LLM inference will be handled remotely via the Groq and Anthropic APIs, eliminating the need for local GPU resources.

8 Team dynamics

The team has four members with skills that match the main parts of the project. Responsibilities are divided clearly, while members will work together during integration and testing.

Abdullah will lead ROS and Gazebo integration for the LLM-controlled robot system. His STRP work gave him experience with LLM-controlled multi-robot systems. His autonomous tow-truck project at Toyota gave him experience with ROS and Gazebo. He is also the only member with prior experience of RAG pipelines and Gazebo.

Salman will lead the design of the simulation environment and contribute to evolutionary optimisation methods. His game-development experience is useful for building the simulation environment, while his AI coursework supports his work on heuristic methods.

Zain and Taha will map taxonomy classes to optimisation models and implement the task-allocation solvers. They have the strongest background in Computational Intelligence and optimisation algorithms. Their work will cover assignment, routing, scheduling, and coalition-formation problems.

All members have completed or are completing Mobile Robotics, giving the team a shared understanding of ROS and multi-robot systems. Abdullah's RAG and Gazebo experience, Salman's design skills, and Zain and Taha's stronger optimisation background cover the main areas in which the other team members have less experience.

9 Tech stack

The system is built on a ROS 2 and Gazebo stack, with MoveIt2 and Nav2 handling arm motion planning and mobile navigation respectively, ros2_control managing robot controllers, and URDF/Xacro and SDF defining robot and world models. The AI planning layer uses the Groq and Anthropic APIs for LLM inference, with Python and NetworkX handling DAG construction and dependency reasoning. Optimization solvers for the seven iTax subclasses are implemented using SciPy, OR-Tools, and PuLP. Knowledge resources including the robot capability registry, skill banks, and world maps are stored as JSON and YAML files. The workspace is built with colcon and orchestrated using tmux and bash scripts.

10 References

References

- [1] Brian P. Gerkey and Maja J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004.
- [2] G. Ayorkor Korsah, Anthony Stentz, and M. Bernardine Dias. A comprehensive taxonomy for multi-robot task allocation. *The International Journal of Robotics Research*, 32(12):1495–1512, 2013.
- [3] Peihan Li, Zijian An, Shams Abrar, and Lifeng Zhou. Large language models for multi-robot systems: A survey, 2026.

Undertaking of Kaavish advisement as an External Supervisor

I hereby affirm that I have read the project details as described on the preceding pages and agree to undertake advisement of this Kaavish project as an External Supervisor. I understand that this role entails the following.

Meeting Meeting the project team regularly, at least once every two weeks, for the entire duration of the Kaavish. The meetings may be held remotely if required.

Advisement Providing supervision and advice to the team in order to ensure steady progress of the project toward its goals.

Liaison Liaising with the Internal Supervisor as required, e.g. to provide feedback or engage in grading.

Other Any other task, depending on availability and suitability, relevant to the Kaavish as communicated by the Internal Supervisor or Kaavish Working Group.

Name: _____

Email: _____

Phone: _____

Designation: _____

Affiliation: _____

Signature: _____